



第4章 指令系统





4.1 指令系统的发展与性能要求





基本概念

- **指令（Instruction）** 是指要计算机执行某种操作的命令
- 从计算机组成的层次结构来说，计算机的指令有：
 - ◆ 微指令：微程序级的命令，它属于硬件
 - ◆ 宏指令：由若干条机器指令组成的软件指令，它属于软件
 - ◆ 机器指令：介于微指令与宏指令之间，每条指令可完成一个独立的算术运算或逻辑运算
- **指令系统ISA（Instruction Set Architecture）** 也称指令集架构
 - ◆ 一台计算机中所有机器指令的集合
 - ◆ 定义机器级程序的格式和行为，以及处理器状态
 - ◆ 指令的格式与功能不仅影响到机器的硬件结构，也影响到系统软件，影响到机器的适用范围





计算机指令系统的发展过程

- 50年代：指令系统只有定点加减、逻辑运算、数据传送、转移等十几至几十条指令
- 60年代后期
 - ◆ 增加了乘除运算、浮点运算、十进制运算、字符串处理等指令，指令数目多达一二百条，寻址方式也趋多样化
 - ◆ 出现系列计算机（指基本指令系统相同、基本体系结构相同的一系列计算机）
- 70年代末期
 - ◆ 大多数计算机的指令系统多达几百条。这种计算机称为复杂指令集计算机（Complex Instruction Set Computer, CISC）
 - ◆ 相对于CISC，又出现了便于VLSI技术实现的精简指令集计算机（Reduced Instruction Set Computer, RISC）





指令系统的要求（1）

- 完备性：指令系统丰富、功能齐全、使用方便
- 有效性：编写的程序能够高效率的运行
 - ◆ 程序占据存储空间小、执行速度快
- 规整性
 - ◆ 对称性：在指令系统中所有寄存器和存储器单元都可同等对待，所有指令都可使用各种寻址方式
 - ◆ 匀齐性：指一种操作性质的指令可以支持各种数据类型
 - ◆ 指令格式和数据格式的一致性：指令长度和数据长度有一定的关系，以方便处理和存取。如，均为字节长度的整数倍





指令系统的要求（2）

■ 兼容性

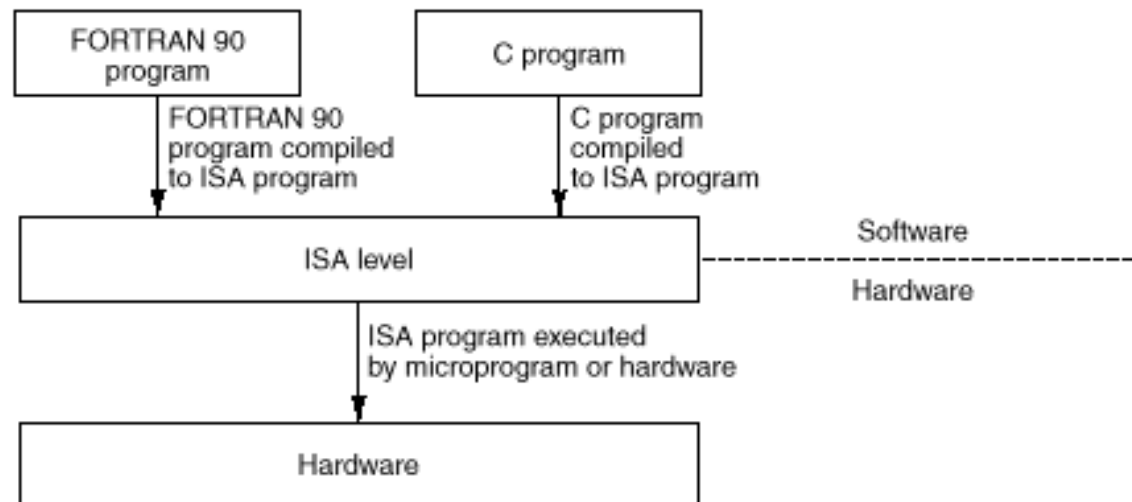
- ◆ 系列机各机种之间具有相同的基本结构和共同的基本指令集，即指令系统是兼容的，各机种上基本软件可以通用
- ◆ 由于不同机种推出的时间不同，在结构和性能上有差异，做到所有软件都完全兼容是不可能的，通常要做到**向后兼容**（**Backward Compatibility**），即在旧处理器上运行的软件仍然可以在新处理器上运行





低级语言与硬件结构的关系

- 高级语言：如C，Java等，其语句和用法与具体机器的指令系统无关
- 低级语言：是面向机器的语言，和具体机器的指令系统密切相关
 - ◆ 机器语言（二进制语言），用指令代码编写程序
 - ◆ 汇编语言（符号语言），用指令助记符来编写程序





高级语言与低级语言比较

比较内容	高级语言	低级语言
对程序员的训练要求 (1) 通用算法 (2) 语言规则 (3) 硬件知识	有 较少 不要	有 较多 要
对机器独立的程度	独立	不独立
编制程序的难易程度	易	难
编制程序所需时间	短	较长
程序执行时间	较长	短
编译过程中对计算机资源的要求	多	少





4.2 指令格式



基本概念

- 指令字（简称指令）即表示一条指令的机器字
- 指令格式是指令字用二进制表示的结构形式
- 指令的构成
 - ◆ 操作码字段：表示指令的操作与功能
 - ◆ 地址码字段：指定参与操作的操作数的地址

操作码字段OP

地址码字段A





操作码

- 指令系统的每一条指令都有一个操作码，它表示该指令应进行什么样的操作
- 操作码字段的位数一般取决于计算机指令系统的规模。例如：
 - ◆ 如果某指令系统只有8条指令，则用3位操作码就可表示
 - ◆ 如果有32条指令，那么就需要5位操作码
- 对于一个机器的指令系统，指令字中操作码字段可以是等长的，也可以是变长的





地址码 (1)

- 按照一条指令中有几个操作数地址，可将该指令称为几操作数指令或几地址指令

◆ 三地址指令

操作码	A1	A2	A3
-----	----	----	----

◆ 二地址指令

操作码	A1	A2
-----	----	----

◆ 一地址指令

操作码	A
-----	---

◆ 零地址指令

操作码	
-----	--





地址码 (2)

■ 零地址指令

- ◆ 指令字中只有操作码，而没有地址码

■ 一地址指令

- ◆ 称为单操作数指令，指令中只有一个操作数
- ◆ 或者是，指令中默认以运算器中累加寄存器AC中的数为操作数1，而地址码字段所指明的数为操作数2，操作结果又放回累加寄存器AC中





地址码 (3)

■ 二地址指令常称为双操作数指令

- ◆ 它的两个地址码字段分别指明参与操作的两个数在内存中或运算器中通用寄存器的地址，A1作存放操作结果的地址

$(A1) \text{ OP } (A2) \rightarrow A1$

■ 三地址指令

- ◆ 指令字中有三个操作数地址

$(A1) \text{ OP } (A2) \rightarrow A3$

- ◆ A1为源操作数1的地址；
- ◆ A2为源操作数2的地址；
- ◆ A3为目的操作数的地址。
- ◆ 其中，A1、A2、A3可以是内存中的单元地址，也可以是运算器中通用寄存器的地址





指令的分类

- 按照各操作数的物理位置不同，可分为三种类型：
 - ◆ 存储器-存储器（SS）型指令：参与操作的数都放在内存里，因此执行这种指令需多次访问内存
 - ◆ 寄存器-寄存器（RR）型指令：从寄存器中取操作数，把操作结果放到另一寄存器。机器执行寄存器-寄存器型指令的速度很快，因为执行这类指令，不需要访问内存，但需要多个寄存器
 - ◆ 寄存器-存储器（RS）型指令：执行此类指令时，既要访问内存单元，又要访问寄存器





指令字长度（1）

- 指令字长度：一个指令字中包含二进制代码的位数
- 机器字长：计算机能直接处理的二进制数据的位数，它决定了计算机的运算精度
 - ◆ 机器字长度通常与寄存器的位数一致
- 指令分类
 - ◆ 单字长指令：指令字长度等于机器字长度
 - ◆ 半字长指令：指令字长度等于半个机器字长度
 - ◆ 双字长指令：指令字长度等于两个机器字长度





指令字长度（2）

- 使用多字长指令，目的在于提供更大的地址空间
 - ◆ 但缺点是必须两次或多次访问内存以取出一整条指令，降低了CPU的处理速度
- **等长**指令字结构：各种指令字长度是相等的。这种指令字结构简单，指令字长度是不变的
- **变长**指令字结构：各种指令字长度随指令功能而异。使用频度高的指令字较短，代码密度高，指令的控制较复杂





扩展操作码指令格式

	OP	A ₁	A ₂	A ₃	
4 位操作码	0000	A ₁	A ₂	A ₃	最多15条三地址指令
	0001	A ₁	A ₂	A ₃	
	⋮	⋮	⋮	⋮	
	1110	A ₁	A ₂	A ₃	
8 位操作码	1111	0000	A ₂	A ₃	最多15条二地址指令
	1111	0001	A ₂	A ₃	
	⋮	⋮	⋮	⋮	
	1111	1110	A ₂	A ₃	
12 位操作码	1111	1111	0000	A ₃	最多15条一地址指令
	1111	1111	0001	A ₃	
	⋮	⋮	⋮	⋮	
	1111	1111	1110	A ₃	
16 位操作码	1111	1111	1111	0000	16条零地址指令
	1111	1111	1111	0001	
	⋮	⋮	⋮	⋮	
	1111	1111	1111	1111	





指令助记符

- 由于硬件只能识别1和0，所以采用二进制操作码是必要的，但不适合人阅读和书写程序
- 因此，每条指令通常用3个或4个英文缩写字母来表示。这种缩写码叫做指令助记符





典型指令助记符

典型指令	指令助记符	二进制操作码
加法	ADD	001
减法	SUB	010
传送	MOV	011
跳转	JMP	100
转子	JSR	101
存储	STR	110
读数	LAD	111

- 注意：在不同的计算机中，指令助记符的规定是不一样的。把人认识的指令助记符转换成与之对应的二进制码后机器才能“理解”。这种转换借助汇编器自动完成，汇编器相当于一个“翻译”





举例1:八位微型计算机的指令格式

- 8位微型机字长只有8位，指令结构是一种可变字长形式，包含单字长、双字长、三字长指令等多种格式
- 内存按字节编址

单字长指令

操作码

双字长指令

操作码

操作数地址

三字长指令

操作码

操作数地址

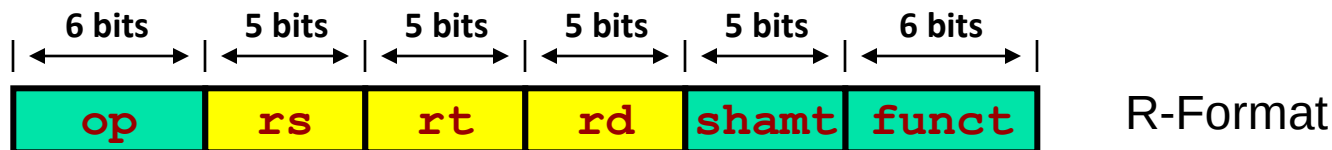
操作数地址





举例2: MIPS R4000指令格式

- 80年代后期推出RISC, 字长32位, 字节寻址, 指令32位等长



OP字段: 操作码

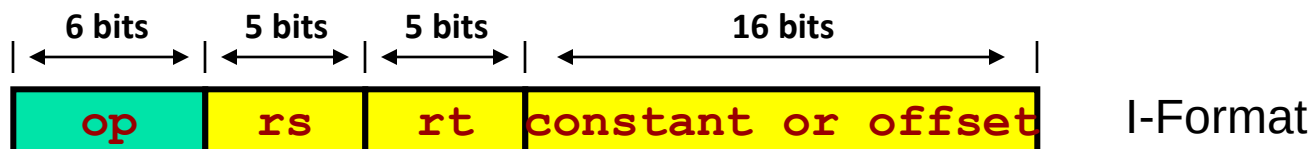
rs字段: 第1个源操作数寄存器

rt字段: 第2个源操作数寄存器

rd字段: 用于存放操作结果目的寄存器

shamt字段: 移位值, 用于移位指令

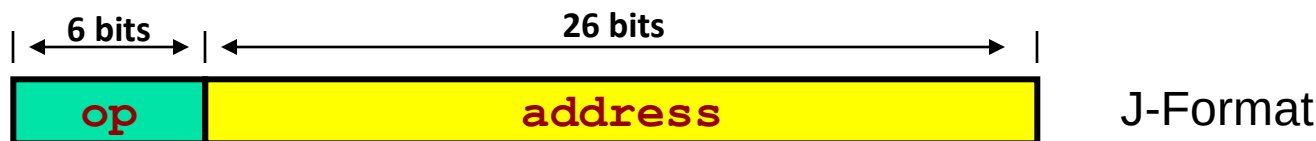
funct字段: 功能码



用于加立即数或取数、存数指令

16位offset字段: 用于取数或存数指令确定操作数在内存的地址,
即: 基址寄存器rs的内容+offset (16位有符号数)

16位constant字段: 用于加立即数指令中存放立即数 (16位有符号数)



转跳指令

26位address字段: 将address×4替换PC的低28位形成32位转跳目标地址





MIPS指令的字段值

■ R格式、I格式和J格式指令字段值（十进制）

指令	格式	OP	rs	rt	rd	shamt	funct	说明
add（加）	R	0	reg	reg	reg	0	32	加法
sub（减）	R	0	reg	reg	reg	0	34	减法
加立即数	I	8	reg	reg	constant			常数
lw（取字）	I	35	reg	reg	address			16位地址
sw（存字）	I	43	reg	reg	address			16位地址
J（转跳）	J	2	address					26位地址

◆ **reg**表示寄存器号，取值为0 ~ 31

◆ **funct**功能码，这里指明是加法还是减法





R格式指令举例

Assembly Code

```
add $s0, $s1, $s2
```

```
sub $t0, $t3, $t5
```

Field Values

op	rs	rt	rd	shamt	funct
0	17	18	16	0	32
0	11	13	8	0	34
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Machine Code

op	rs	rt	rd	shamt	funct	
000000	10001	10010	10000	00000	100000	(0x02328020)
000000	01011	01101	01000	00000	100010	(0x016D4022)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

寄存器名	寄存器号
\$s0 ~ \$s7	16 ~ 23
\$t0 ~ \$t7	8 ~ 15

Note the order of registers in the assembly code:

```
add rd, rs, rt;  rs+rt->rd
```





I格式指令举例

Assembly Code

```
addi $s0, $s1, 5
```

```
addi $t0, $s3, -12
```

```
lw    $t2, 32($0)
```

```
sw    $s1, 4($t1)
```

Field Values

op	rs	rt	imm
8	17	16	5
8	19	8	-12
35	0	10	32
43	9	17	4

6 bits

5 bits

5 bits

16 bits

寄存器名

寄存器号

\$s0 ~ \$s7

16 ~ 23

\$t0 ~ \$t7

8 ~ 15

Note the differing order of registers in assembly and machine codes:

```
addi rt, rs, imm
```

```
lw    rt, imm(rs)
```

```
sw    rt, imm(rs)
```

Machine Code

op	rs	rt	imm	
001000	10001	10000	0000 0000 0000 0101	(0x22300005)
001000	10011	01000	1111 1111 1111 0100	(0x2268FFF4)
100011	00000	01010	0000 0000 0010 0000	(0x8C0A0020)
101011	01001	10001	0000 0000 0000 0100	(0xAD310004)

6 bits

5 bits

5 bits

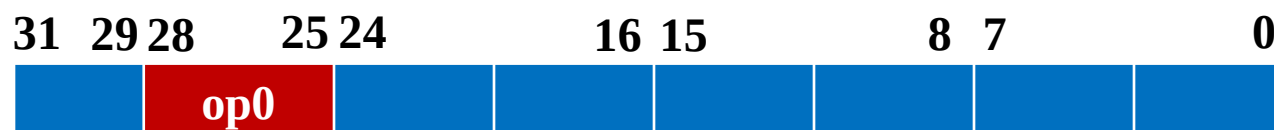
16 bits





举例3： A64指令格式（1）

■ A64 instruction set encoding



字段编码 op0	编码分组或指令类型
0000	保留（Reserved）
0001	未分配（Unallocated）
0010	SVE（Scalable Vector Extensio）指令
0011	未分配（Unallocated）
100x	数据处理 — 立即寻址
101x	分支、异常发生和系统指令
x1x0	Loads and Stores内存访问指令
x101	数据处理 — 寄存器
x111	数据处理 — 浮点与SIMD指令





举例3: A64指令格式 (2)

■ add/subtract (immediate) instruction set encoding

31 30 29 28 25 24 23 22 21 20 10 9 4 0

sf op S 1 0 0 0 1 0 sh imm12 Rn(5bit) Rd(5bit)

Decode fields	Instruction
sf op S	
0 0 0	ADD (immediate) - 32-bit
0 0 1	ADDS (immediate) - 32-bit
0 1 0	SUB (immediate) - 32-bit
0 1 1	SUBS (immediate) - 32-bit
1 0 0	ADD (immediate) - 64-bit
1 0 1	ADDS (immediate) - 64-bit
1 1 0	SUB (immediate) - 64-bit
1 1 1	SUBS (immediate) - 64-bit

op	Instruction
00x	PC-rel. addressing
010	Add/subtract(immediate)
011	Add/subtract(imm., tags)
100	Logical (immediate)
101	Move wide (immediate)
110	Bitfield
111	Extract

例: add x16, x16, #0xff8 @ (x16) + 0xff8 $\Rightarrow$ x16

机器码为: 1001 0001 0011 1111 1110 0010 0001 0000

Rn:源寄存器

Rd:目标寄存器

imm12:12位立即数

S:为1时指令执行将影响
PSTATE中的标志位NZCV





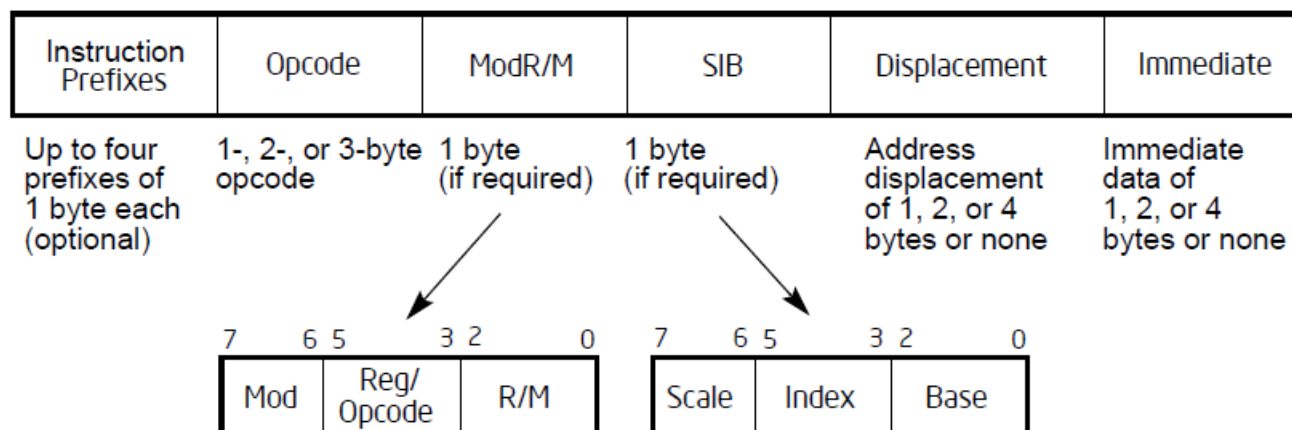
举例4: Pentium指令格式

- 指令字长度是可变的，80386从 1 字节到15字节
- Pentium指令是典型的CISC指令，之所以选择可变长，主要是为了向后兼容
- 指令的前缀是可选项，其作用是对其后的指令本身进行显示约定，每个前缀占 1 个字节





Intel64和IA-32架构指令格式



REX前缀字节用于64位模式中

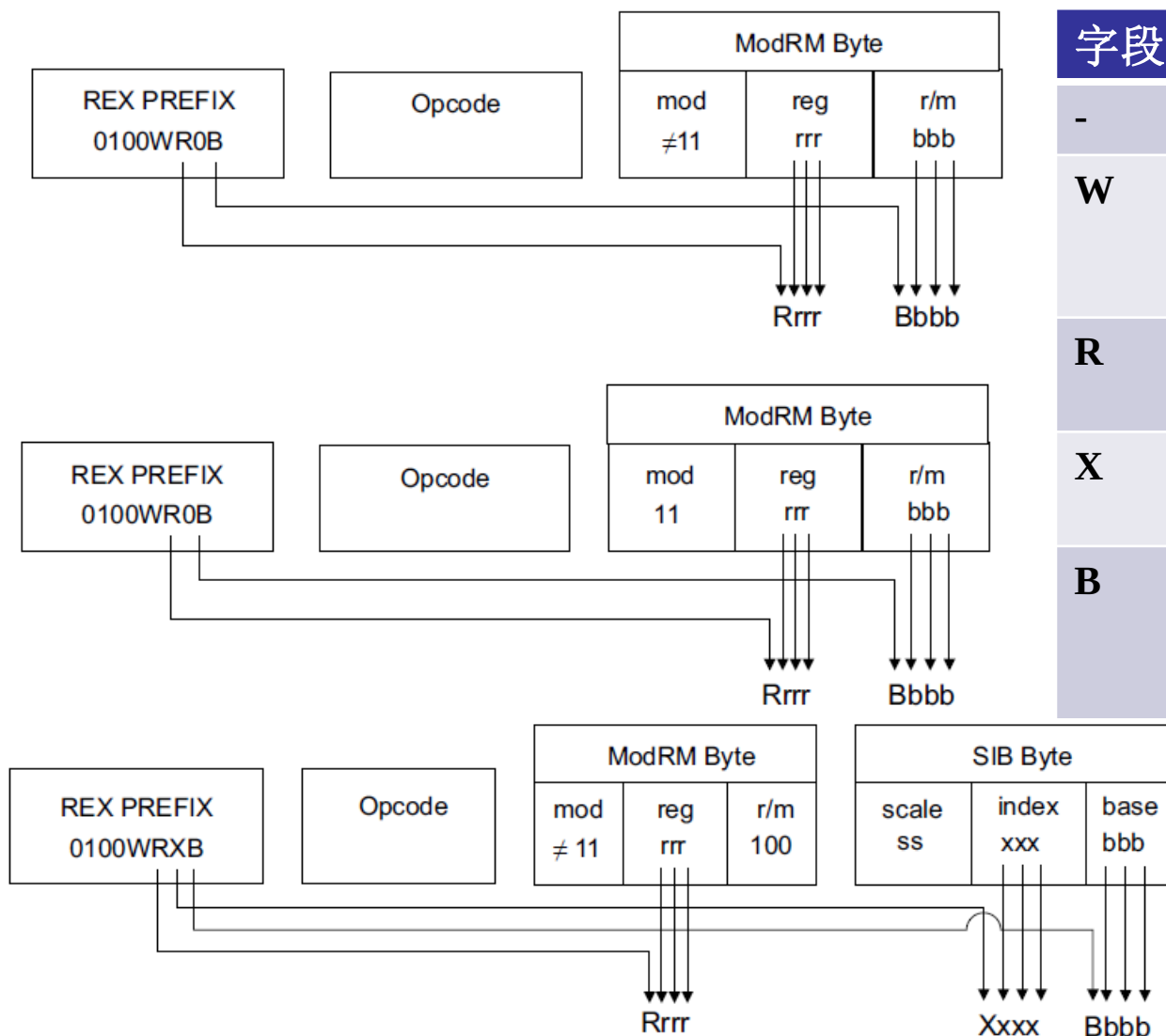
- 说明64位操作数宽度
- 指定扩展的寄存器

Legacy Prefixes	REX Prefix	Opcode	ModR/M	SIB	Displacement	Immediate
Grp 1, Grp 2, Grp 3, Grp 4 (optional)	(optional)	1-, 2-, or 3-byte opcode	1 byte (if required)	1 byte (if required)	Address displacement of 1, 2, or 4 bytes	Immediate data of 1, 2, or 4 bytes or none





REX前缀中的R和B用于指定r8~r15



字段名	位	定义
-	7:4	0100
W	3	0=CS.D确定操作数宽度 1=64位操作数
R	2	ModR/m中reg字段的扩展位
X	1	SIB中index字段的扩展位
B	0	ModR/m中reg字段、SIB中base字段或操作码中reg字段的扩展位





RISC vs. CISC

地址	机器码	指令	地址	机器码	指令
000000000400ef4	<main>:		000000000400da0	<main>:	
400ef4:	a9be7bfd	stp x29, x30, [sp, #-32]!	400da0:	53	push %rbx
400ef8:	910003fd	mov x29, sp	400da1:	83 ff 01	cmp \$0x1,%edi
400efc:	7100041f	cmp w0, #0x1	400da4:	75 10	jne 400db6 <main+0x16>
400f00:	54000700	b.eq 400fe0 <main+0xec> // b.none	400da6:	48 8b 05 9b 29 20 00	mov 0x20299b(%rip),%rax
400f04:	f9000bb3	str x19, [x29, #16]	400dad:	48 89 05 b4 29 20 00	mov %rax,0x2029b4(%rip)
400f08:	aa0103f3	mov x19, x1	400db4:	eb 63	jmp 400e19 <main+0x79>
400f0c:	7100081f	cmp w0, #0x2	400db6:	48 89 f3	mov %rsi,%rbx
400f10:	54000801	b.ne 401010 <main+0x11c> // b.any	400db9:	83 ff 02	cmp \$0x2,%edi
400f14:	d0000001	adrp x1, 402000 <submitr+0x40c>	400dbc:	75 3a	jne 400df8 <main+0x58>
400f18:	9113c021	add x1, x1, #0x4f0	400dbe:	48 8b 7e 08	mov 0x8(%rsi),%rdi
400f1c:	f9400660	ldr x0, [x19, #8]	400dc2:	be b4 22 40 00	mov \$0x4022b4,%esi
400f20:	97ffff48	bl 400c40 <fopen@plt>	400dc7:	e8 44 fe ff ff	callq 400c10 <fopen@plt>
400f24:	90000101	adrp x1, 420000 <strlen@GLIBC_2.17>	400dcc:	48 89 05 95 29 20 00	mov %rax,0x202995(%rip)
400f28:	f9039020	str x0, [x1, #1824]	400dd3:	48 85 c0	test %rax,%rax
400f2c:	b4000640	cbz x0, 400ff4 <main+0x100>	400dd6:	75 41	jne 400e19 <main+0x79>
400f30:	f9400bb3	ldr x19, [x29, #16]	400dd8:	48 8b 4b 08	mov 0x8(%rbx),%rcx
400f34:	940001bc	bl 401624 <initialize_bomb>	400ddc:	48 8b 13	mov (%rbx),%rdx
400f38:	d0000000	adrp x0, 402000 <submitr+0x40c>	400ddf:	be b6 22 40 00	mov \$0x4022b6,%esi
400f3c:	9114e000	add x0, x0, #0x538	400de4:	bf 01 00 00 00	mov \$0x1,%edi
400f40:	97ffff64	bl 400cd0 <puts@plt>	400de9:	e8 12 fe ff ff	callq 400c00 <__printf_chk@plt>
400f44:	d0000000	adrp x0, 402000 <submitr+0x40c>	400dee:	bf 08 00 00 00	mov \$0x8,%edi
400f48:	9115e000	add x0, x0, #0x578	400df3:	e8 28 fe ff ff	callq 400c20 <exit@plt>
400f4c:	97ffff61	bl 400cd0 <puts@plt>	400df8:	48 8b 16	mov (%rsi),%rdx
400f50:	9400026f	bl 40190c <read_line>	400dfb:	be d3 22 40 00	mov \$0x4022d3,%esi
400f54:	94000035	bl 401028 <phase_1>	400e00:	bf 01 00 00 00	mov \$0x1,%edi
400f58:	940002b8	bl 401a38 <phase_defused>	400e05:	b8 00 00 00 00	mov \$0x0,%eax
400f5c:	d0000000	adrp x0, 402000 <submitr+0x40c>	400e0a:	e8 f1 fd ff ff	callq 400c00 <__printf_chk@plt>
400f60:	9116a000	add x0, x0, #0x5a8	400e0f:	bf 08 00 00 00	mov \$0x8,%edi
400f64:	97ffff5b	bl 400cd0 <puts@plt>	400e14:	e8 07 fe ff ff	callq 400c20 <exit@plt>
400f68:	94000269	bl 40190c <read_line>	400e19:	e8 84 05 00 00	callq 4013a2 <initialize_bomb>
400f6c:	94000039	bl 401050 <phase_2>	400e1e:	bf 38 23 40 00	mov \$0x402338,%edi
400f70:	940002b2	bl 401a38 <phase_defused>	400e23:	e8 e8 fc ff ff	callq 400b10 <puts@plt>
400f74:	d0000000	adrp x0, 402000 <submitr+0x40c>	400e28:	bf 78 23 40 00	mov \$0x402378,%edi
400f78:	91176000	add x0, x0, #0x5d8	400e2d:	e8 de fc ff ff	callq 400b10 <puts@plt>
400f7c:	97ffff55	bl 400cd0 <puts@plt>	400e32:	e8 67 06 00 00	callq 40149e <read_line>
400f80:	94000263	bl 40190c <read_line>	400e37:	48 89 c7	mov %rax,%rdi
400f84:	94000050	bl 4010c4 <phase_3>	400e3a:	e8 a1 00 00 00	callq 400ee0 <phase_1>
			400e3f:	e8 80 07 00 00	callq 4015c4 <phase_defused>

鲲鹏920处理器

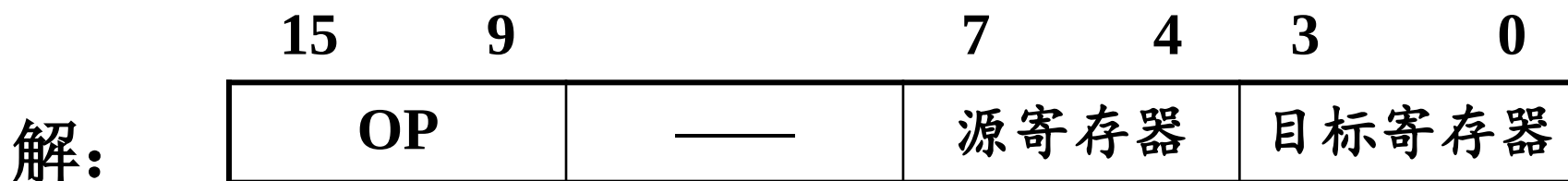
x86-64处理器





例1

指令格式如下所示，其中**OP**为操作码，试分析指令格式的特点



(1) 16位长二地址指令

(2) 操作码字段**OP**可以指定128条指令

(3) 源寄存器和目标寄存器都是通用寄存器（可分别指定16个），所以是**RR**型指令，两个操作数均在寄存器中

(4) 这种指令结构常用于算术逻辑运算类指令





4.3 指令和数据的寻址方式



基本概念（1）

- 存储器既可用来存放数据，又可用来存放指令
- 操作数或指令在存储器中的地址：某个操作数或某条指令存放在某个存储单元时，其存储单元的编号
- 在存储器中，写入或读出操作数/指令字的方式有：
 - ◆ 地址指定方式
 - ◆ 相联存储方式（按内容寻址方式）
 - ◆ 堆栈存取方式



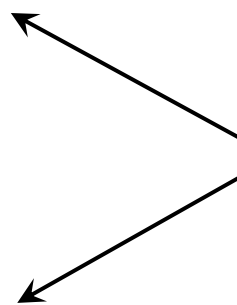
几乎所有的计算机
中均采用这种方式





基本概念（2）

- 当采用地址指定方式时，形成操作数或指令地址的方式，称为寻址方式
- 寻址方式分为两类
 - ◆ 指令寻址方式
 - 顺序寻址方式
 - 跳跃寻址方式
 - ◆ 数据寻址方式



前者比较简单
后者比较复杂

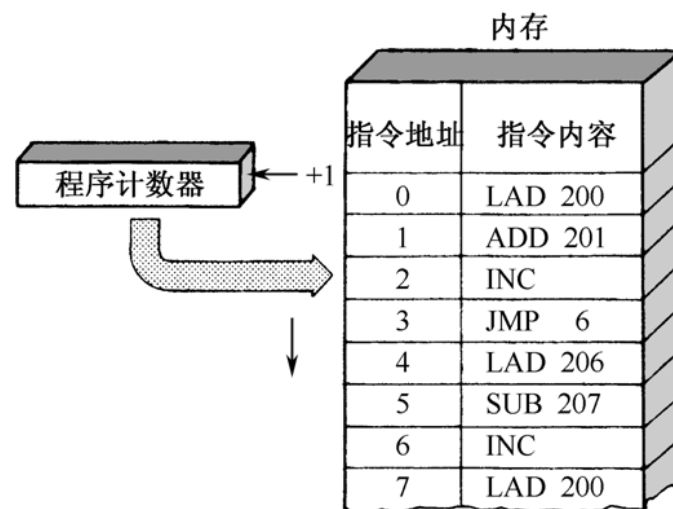




一、指令寻址方式

■ 顺序寻址方式

- ◆ 指令地址在内存中按顺序安排，当执行一段程序时，通常是一条指令接一条指令的顺序执行
- ◆ 从存储器取出第一条指令，然后执行这条指令；接着从存储器取出第二条指令，在执行第二条指令；接着再取出第三条指令……这种程序顺序执行的过程，称之为指令的顺序寻址方式
- ◆ 使用程序计数器（又称指令指针寄存器）PC来计数指令的顺序号，该顺序号就是指令在内存中的地址



(a) 指令的顺序寻址方式

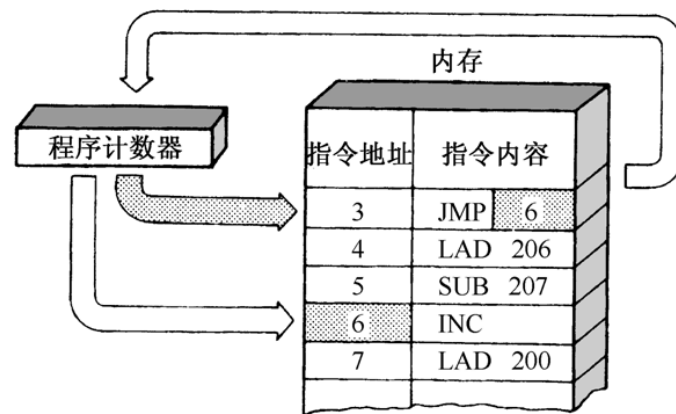




指令寻址方式（续）

■ 跳跃寻址方式

- ◆ 所谓跳跃寻址，是指下条指令的地址码不是由程序计数器给出，而是由本条指令给出
- ◆ 程序跳跃后，按新的指令地址开始顺序执行，指令计数器的内容也必须相应改变，以便及时跟踪新的指令地址
- ◆ 采用跳跃寻址方式，可以实现程序转移或构成循环程序，从而能缩短程序长度，或将某些程序作为公共程序引用
- ◆ 指令系统中的各种条件转移、无条件转移和子程序调用等指令，就是为了实现指令的跳跃寻址而设置的



(b) 指令的跳跃寻址方式





二、操作数寻址方式

- 形成操作数有效地址的方法，称为操作数寻址方式
- 一种单地址指令的结构如下所示，其中用X，I，A各字段组成该指令的操作数地址

操作码(OP)	变址(X)	间址(I)	形式地址(A)
---------	-------	-------	---------

- 指令中操作数字段的地址码是由形式地址（也称偏移量）和寻址方式特征位等组合形成。
 - ◆ 寻址过程就是把操作数的形式地址，变换为操作数的有效地址的过程





1. 隐含寻址

- 在指令中不显式给出而是隐含（默认）给出操作数的地址
- 例如，一种单地址的指令格式，没有在地地址字段中指明第2个操作数地址，而是默认累加寄存器作为第2个操作数地址，所以累加器对单地址指令格式来说是隐含地址

例：mulq (%rbx)

$\text{Reg}[\%rdx]:\text{Reg}[\%rax] \leftarrow \text{Reg}[\%rax] \times \text{Mem}[\text{Reg}[\%rbx]]$





2. 立即寻址

- 指令的地址字段指出的不是操作数的地址，而是**操作数本身**。这种方式的特点是指令执行速度较快，取指令的同时就取到了操作数，不需要再次访存取操作数
- 例如：单地址的移位指令格式为



- 地址字段不是操作数的地址，而是一个操作数

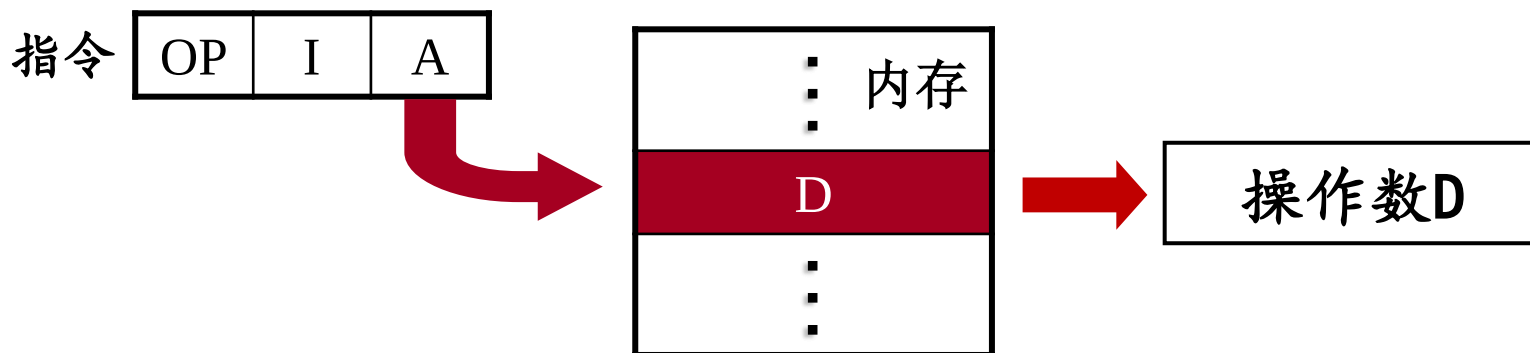
如： `mov $0x6030f0, %edi`





3. 直接寻址

- 在指令地址字段中直接指出操作数在内存的地址A
- 用直接寻址方式时，指令字中的形式地址A就是操作数的有效地址EA。因此通常把形式地址A又称为直接地址
- 如果用D表示操作数，那么直接寻址的逻辑表达式为： $D = (A)$



REX OP

Displacement

movq 0x1122334455, %rax

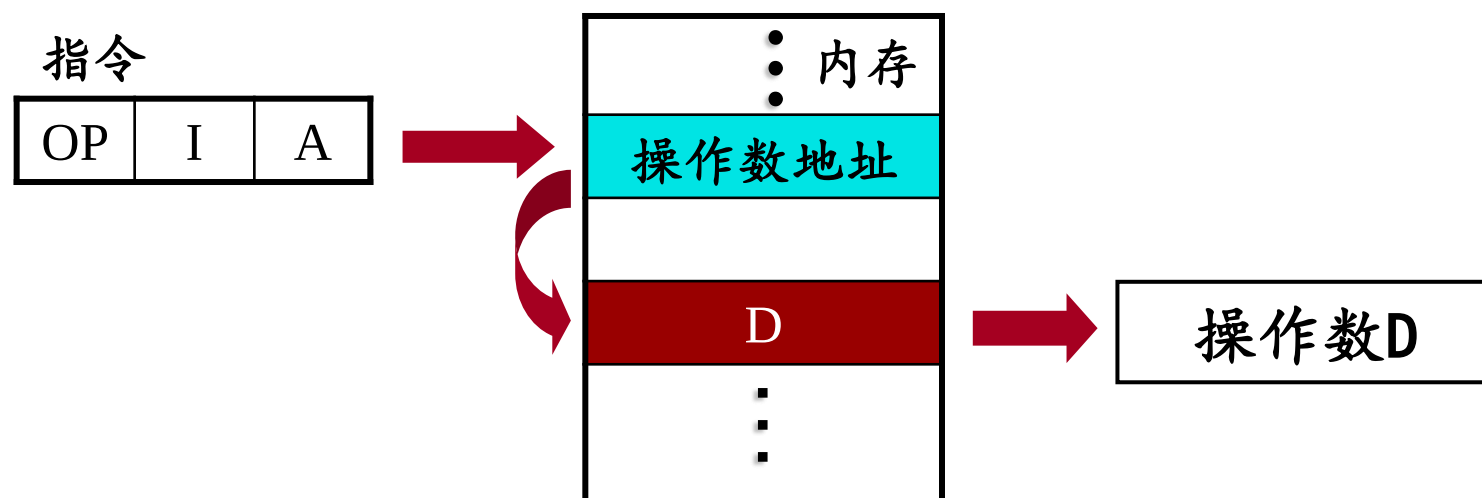
0x48	0xa1	0x55	0x44	0x33	0x22	0x11	0x00	0x00	0x00
------	------	------	------	------	------	------	------	------	------





4. 间接寻址（1）

- 在指令的地址字段中的形式地址**A**不是操作数的真正地址，而是操作数地址的地址
- 间接寻址方式是早期计算机中经常采用的方式，但由于两次访存，执行速度慢，现在已不太使用





间接寻址 (2)

- 把直接寻址和间接寻址结合起来，指令格式如下：

操作码	I	A
-----	---	---

- ◆ 寻址特征位 $I=0$ ，表示直接寻址，有效地址 $EA=A$
- ◆ 寻址特征位 $I=1$ ，表示间接寻址，有效地址 $EA=(A)$



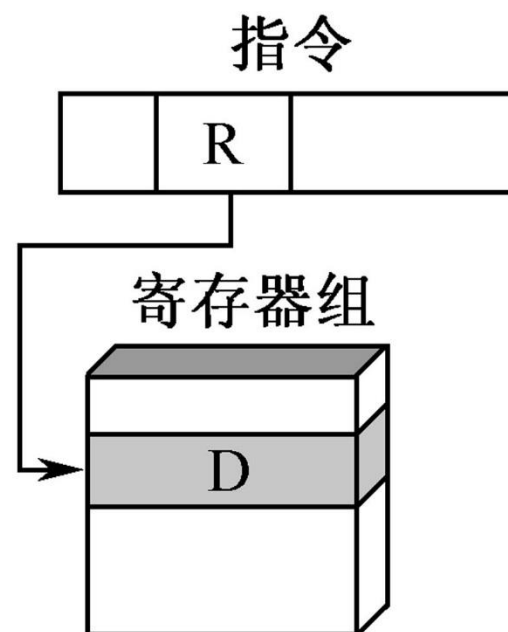
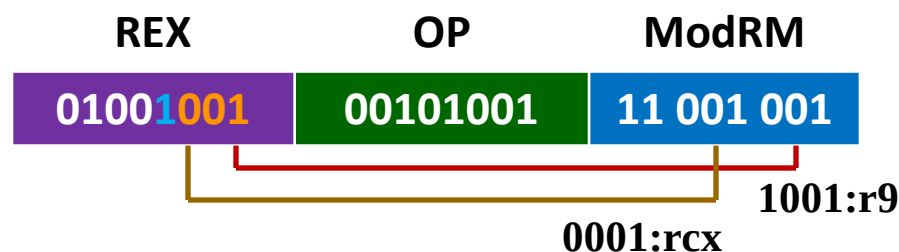


5. 寄存器寻址

■ 寄存器寻址方式

- ◆ 在指令的地址字段中给出的操作数地址不是内存的地址，而是CPU中通用寄存器的**编号**

如： `subq %rcx, %r9`



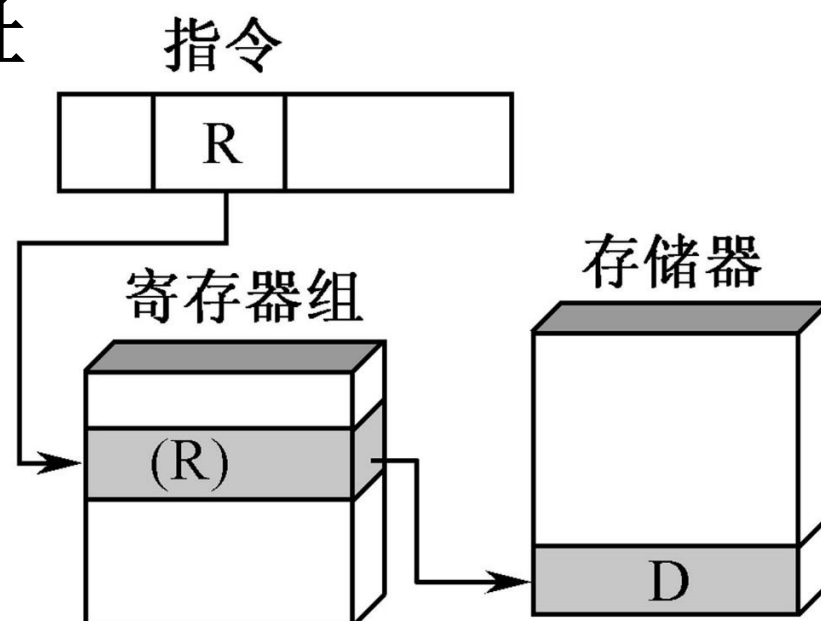
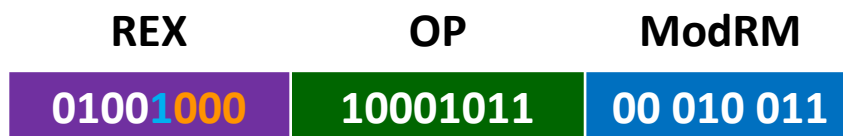


6. 寄存器间接寻址

■ 寄存器间接寻址方式

- ◆ 在指令的地址字段中指定的寄存器的内容不是操作数，而是操作数的地址

如： `mov (%rbx), %rdx`



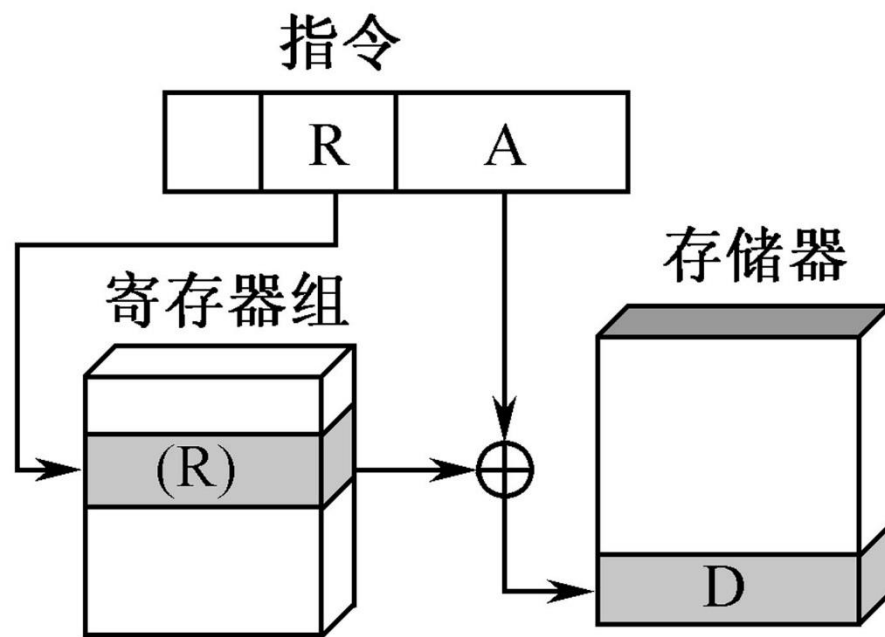
7. 偏移寻址

是直接寻址和寄存器间接寻址方式的结合

$$EA = (R) + A$$

其中：R代表寄存器，A为偏移量（形式地址），
是一个有符号数

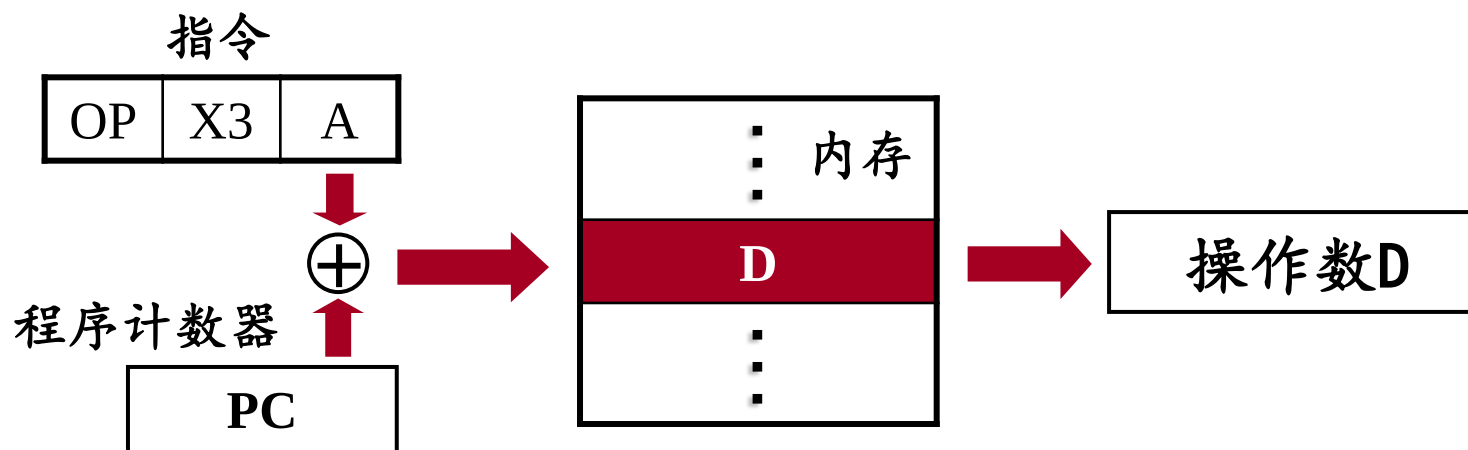
- 相对寻址
- 基址寻址
- 变址寻址





相对寻址

- 程序计数器PC的内容加上指令格式中的偏移量A（有符号数）形成操作数的有效地址
- 程序计数器的内容就是当前指令的地址，“相对”寻址，指的是相对于当前指令地址
- 好处是程序员所编程序可以放在内存任何地方



如: `cmp %rax, 0x2022a9(%rip)`

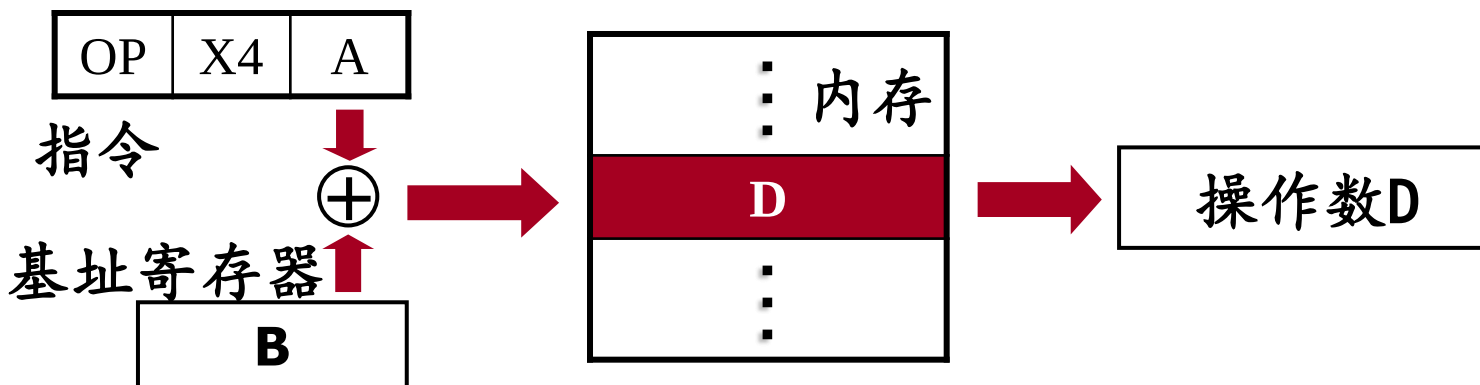
REX	OP	ModRM	Displacement			
0x48	0x39	00 000 101	0xa9	0x22	0x20	0x00





基址寻址

- 将CPU中**基址寄存器**的内容加上指令中的偏移量A得到的操作数有效地址
- 优点是可以扩大寻址能力。基址寄存器的位数可以设置得很长，从而可以在较大的存储空间中寻址



如: `mov %rax, 0x0(%rbp)`

REX	OP	ModRM	Displacement
01001000	10001001	01 000 101	00000000





变址寻址

- 将CPU中**变址寄存器**的内容加上指令中的偏移量A得到的操作数有效地址。与基址寻址方式类似。但变址寄存器的内容**自动**递增或递减。

$$EA = (R) + A$$

$$R \leftarrow (R \pm 1 \text{ 或 } 2)$$

- 使用变址寻址方式的目的在于扩大寻址空间，而在于实现程序块的规律性变化（自动加1减1、或加2减2）

如： `scas %es: (%rdi), %al ;` 执行后 $(\%rdi) \pm 1 \rightarrow \%rdi$

OP

10101110





动手做

某机器字长16位，主存按字节编址，转移指令采用相对寻址，由两个字节组成，第一字节为操作码字段，第二字节为相对位移量字段。假定取指令时，每取一个字节PC自动加1。若某转移指令所在主存地址为2000H，相对位移量字段的内容为06H，则该转移指令成功转以后的目标地址是

A.2006H

B.2007H

C.2008H

D.2009H





动手做

某计算机有16个通用寄存器，采用32位定长指令字，操作码字段（含寻址方式位）为8位，Store指令的源操作数和目的操作数分别采用寄存器直接寻址和基址寻址方式，若基址寄存器可使用任一通用寄存器，且偏移量用补码表示，则Store指令中偏移量的取值范围是

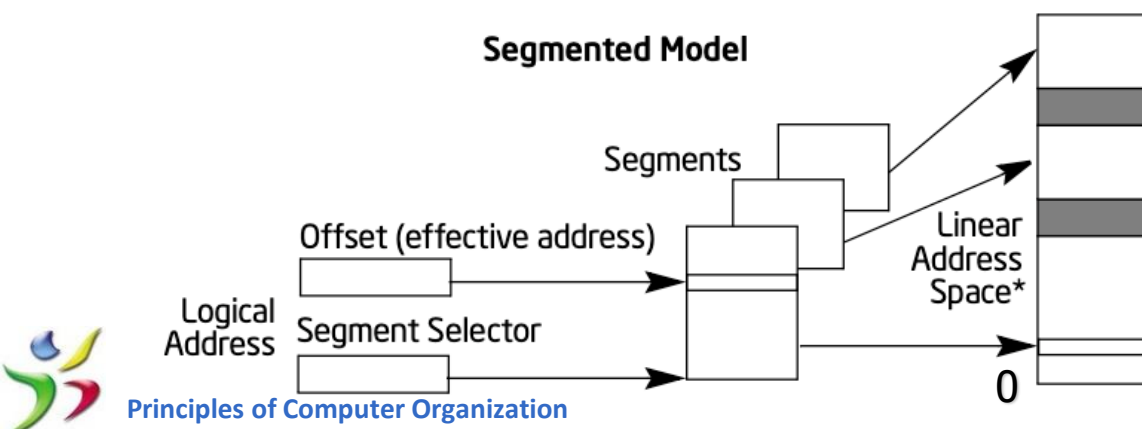
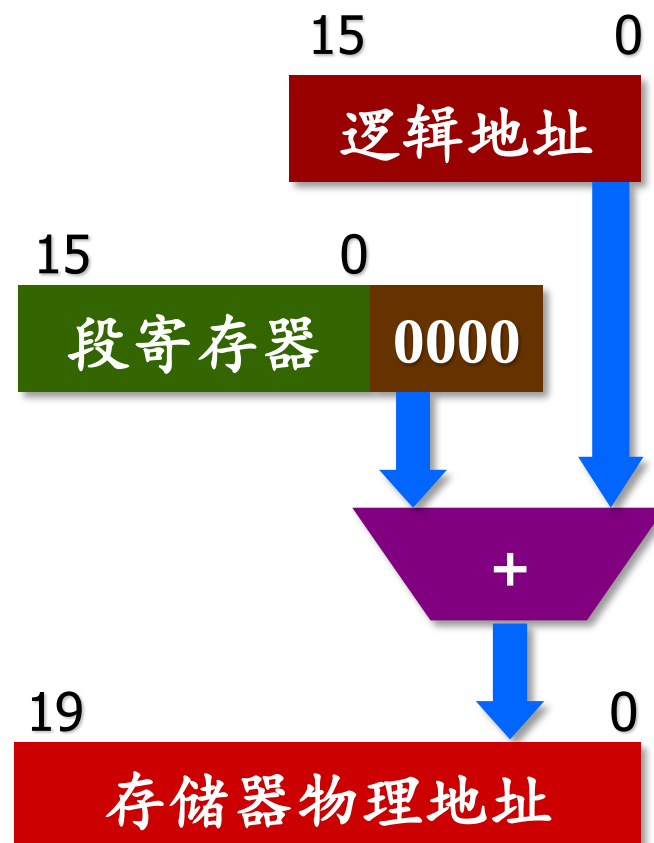
- A -32768~+32767
- B -32767~+32768
- C -65536~+65535
- D -65535~+65536



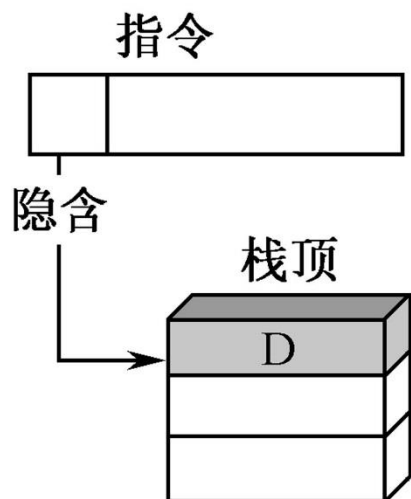


8. 段寻址方式

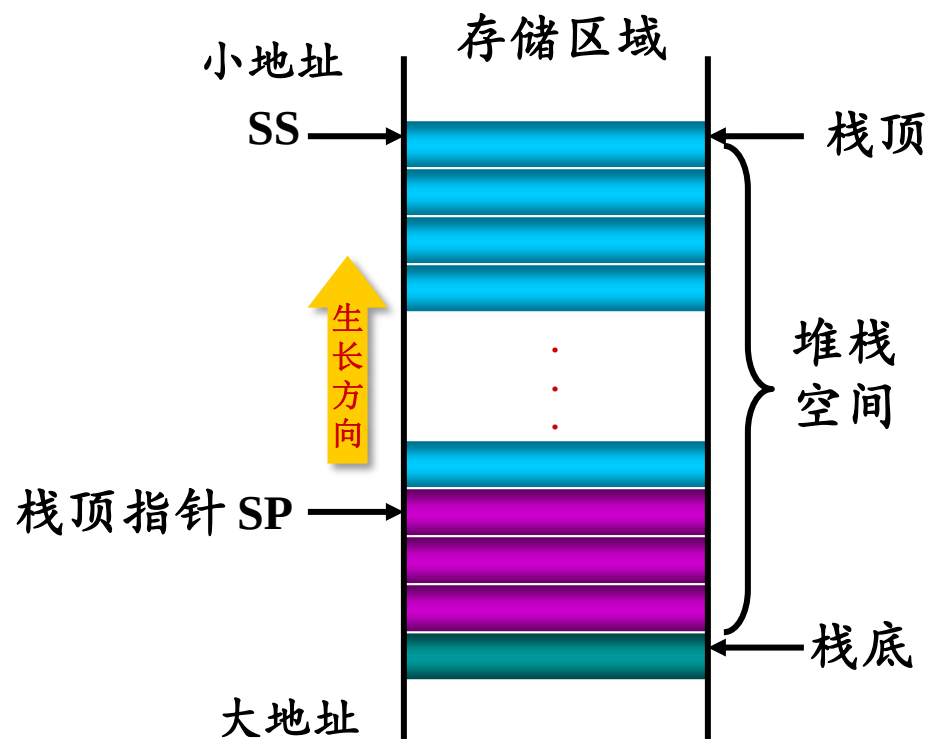
- 某微处理器有20位的地址，可直接寻址1M字节的存储空间
- 将整个1M的存储器以64K为单位划分成若干段
- 寻址时，段寄存器中的16位数左移4位，然后和16位偏移量（逻辑地址）相加，得到20位物理地址



9. 堆栈寻址



寄存器堆栈



存储器堆栈

- 堆栈是一块能存储数据的存储区域，且数据的存取一般只能通过栈顶进行，其特点是“后进先出”，即：
LIFO



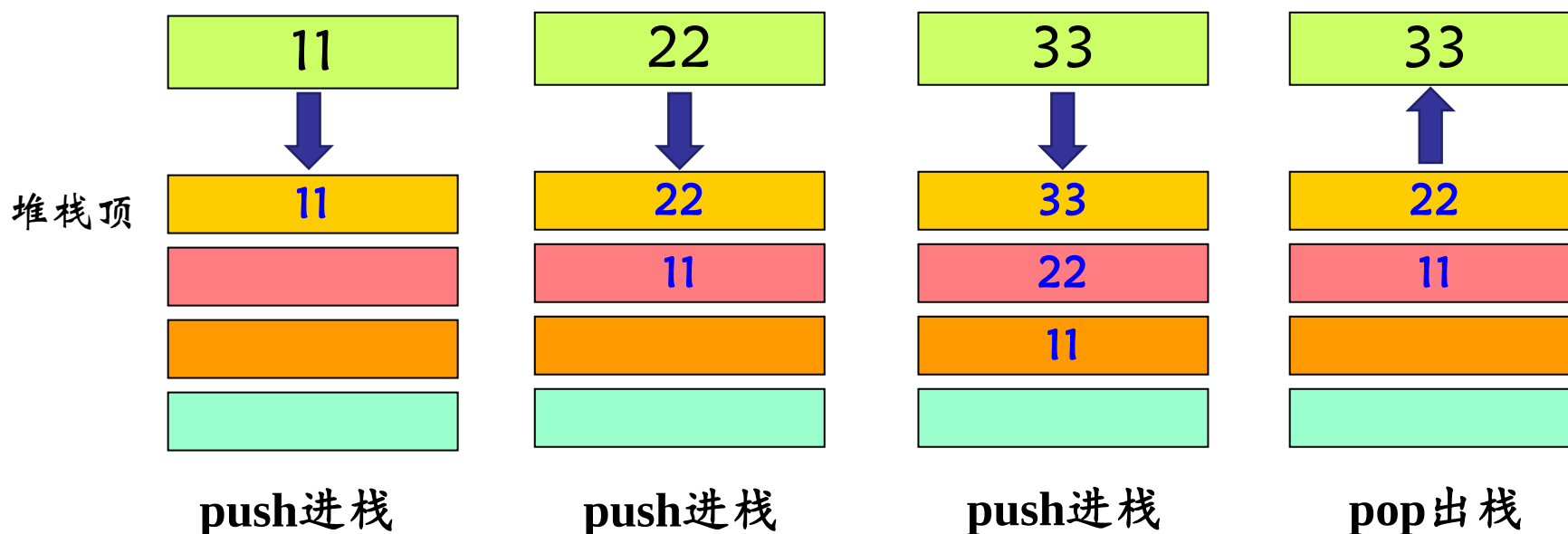
寄存器堆栈

- CPU中有一组专门的寄存器称之为串联堆栈，如有16+个，其中每一个寄存器能保存一个字的数据
- 数据通过栈顶进入或弹出堆栈
- CPU通过“进栈”指令把数据送入堆栈，而通过“出栈”指令把数据从堆栈中取出
- 特点
 - ◆ 入栈和出栈时，栈顶不变，数据移动
 - ◆ 访问速度快
 - ◆ 缺点：
 - 寄存器的数目有限，所以堆栈大小受限
 - 数据的读出是破坏性的





寄存器堆栈 进栈与出栈





存储器堆栈

- 在主存中划分出一个称之为堆栈的存储区域，用一个堆栈指示器来指示当前栈顶的位置，它通常是CPU中一个专用的寄存器（**Stack Point, SP**）
- 比串联堆栈灵活
- 栈顶变化，由**SP**指示，而数据不动
- 优点：
 - ◆ 堆栈可以根据需要设置大小
 - ◆ 堆栈个数可以根据需要而定
 - ◆ 还可以使用任意存储器寻址方式访问堆栈中的数据



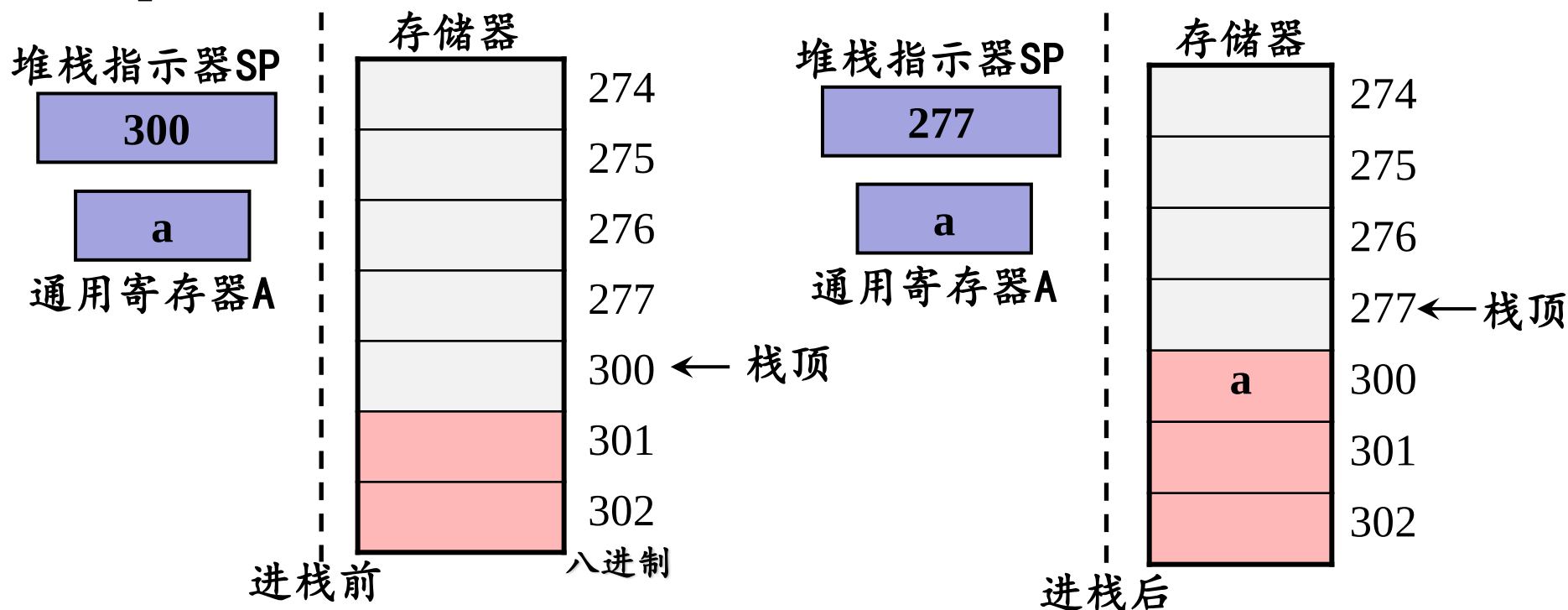


进栈操作

进栈操作:

$$(A) \rightarrow M_{sp}, \quad (SP) - 1 \rightarrow SP$$

其中 (A) 表示通用寄存器 A 的内容, SP 表示堆栈指示器, M_{sp} 表示 SP 所指示的堆栈栈顶单元



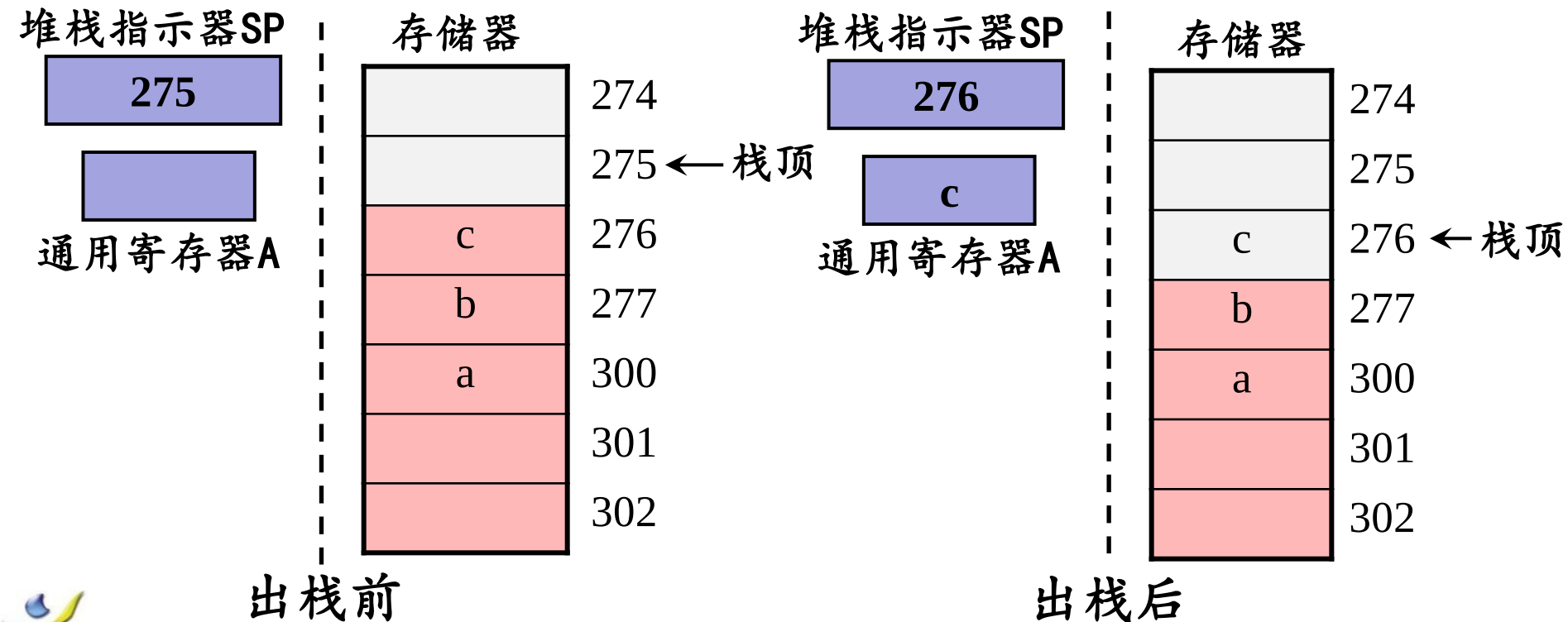


出栈操作

出栈操作:

$$(SP) + 1 \rightarrow SP, (M_{sp}) \rightarrow A$$

其中 (A) 表示通用寄存器 A 的内容, SP 表示堆栈指示器, M_{sp} 表示 SP 所指示的堆栈栈顶单元





说明

在设计处理器的堆栈操作时，有2种选择：

1. 堆栈指示器 SP始终指向空单元

- ◆进栈时，先存入数据，后修改堆栈指示器SP
- ◆出栈时，先修改堆栈指示器SP，然后取出数据
- ◆这里修改SP可以是加操作，也可以是减操作

2. 堆栈指示器 SP始终指向满单元

- ◆进栈时，？自己思考
- ◆出栈时，？自己思考





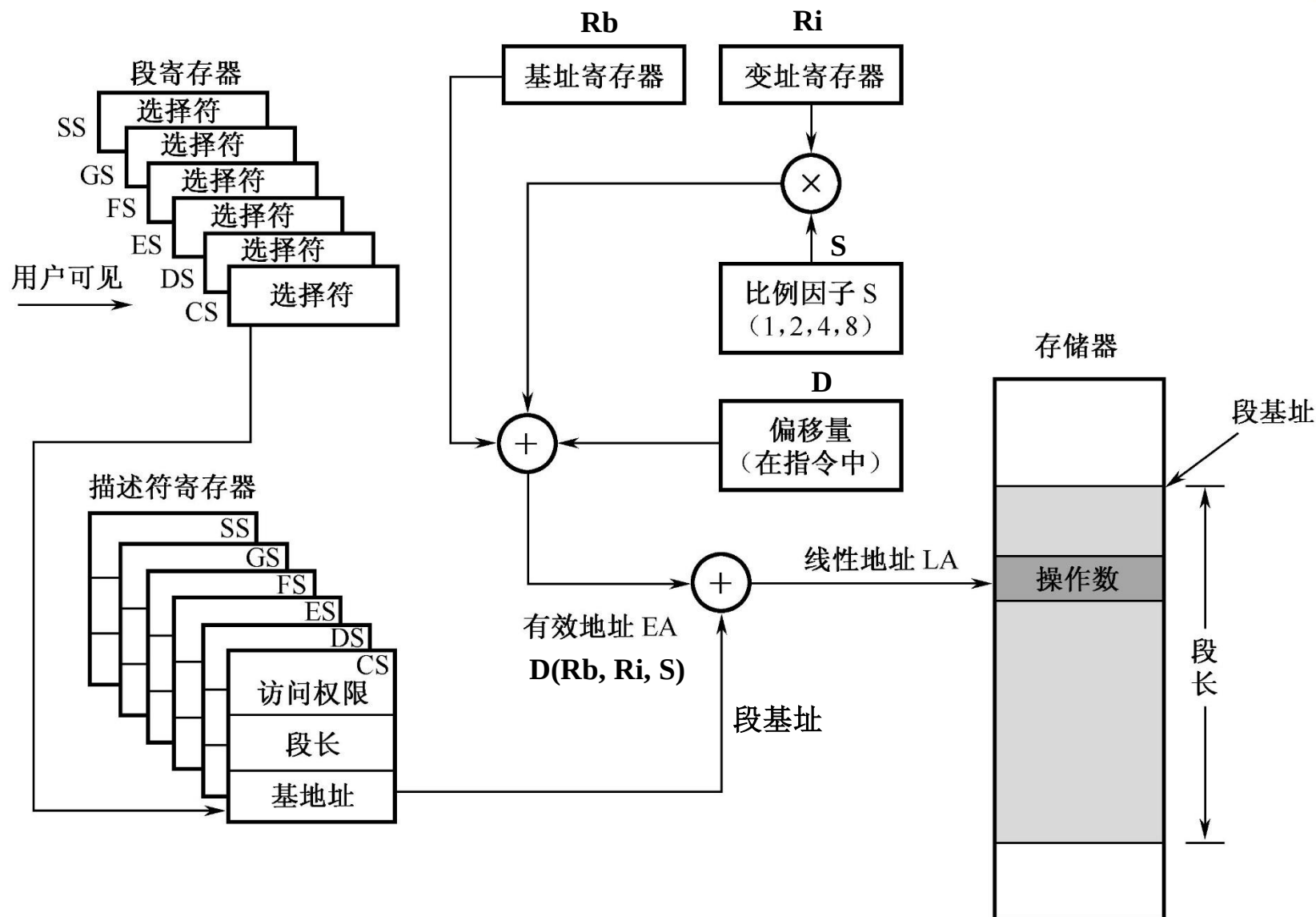
Pentium的寻址方式

- Pentium的外部地址总线宽度是36位，也支持32位物理地址空间
- 实地址模式下，逻辑地址形式为段寻址方式：将段寄存器内容（16位）左移4位，低4位补全0，得到20位段基地址，再加上16位段内偏移，即得20位物理地址
- 在32位操作系统的保护模式下，32位段基地址加上段内偏移得到32位线性地址，由存储管理部件将其转换成32位的物理地址
- Pentium提供了9种有效地址的获取方式
- 各种寻址方式是先得到的有效地址EA（段内偏移），再加上段基地址后才得到物理地址





Pentium各种寻址方式 (32位)





9种寻址方式

(1) 立即寻址

操作数在指令中

(2) 寄存器寻址

操作数在R中

(3) 直接寻址

$$EA = \text{Disp}$$

(4) 基址寻址

$$EA = (Rb)$$

(5) 基址/变址+偏移量寻址

$$EA = (Rb \text{ or } Ri) + \text{Disp}$$

(6) 基址+变址+偏移量寻址

$$EA = (Rb) + (Ri) + \text{Disp}$$

(7) 比例变址+偏移量寻址

$$EA = (Ri) \times S + \text{Disp}$$

(8) 基址+比例变址+偏移量寻址 $EA = (Rb) + (Ri) \times S + \text{Disp}$

(9) 相对寻址

$$EA = (PC) + \text{Disp}$$



$$D(Rb, Ri, S)$$





ARM寻址方式（鲲鹏920处理器）

■ ARMv8 A64处理器六种寻址方式

- ◆ 立即数寻址 `mov x16, #0x2060`
- ◆ 寄存器寻址 `add x2, x19, x0`
- ◆ 寄存器间接寻址 `ldr x17, [x16]`
- ◆ 基址寻址 `str x0, [x1, #8]`
- ◆ 堆栈寻址 `stp x29, x30, [sp, #-16]!    @x30,x29压栈`
 `ldp x29, x30, [sp], #16    @x29,x30出栈`
- ◆ 相对寻址 `adr x0, label    形成相对PC的地址⇒x0`





举例：PDP/11系列机寻址方式

- PDP/11系列机指令字长16位
- 扩展操作码指令格式
- 操作数字段均为6位

指令位 指令类型	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
单操作数指令	操作码（10位）										目标地址(6位)					
双操作数指令	操作码(4位)				源地址(6位)						目标地址(6位)					
转移指令	操作码（8位）								位移量（8位）							
转子指令	操作码（7位）							寄存器号								
子程序返回指令	操作码（13位）															
条件码操作指令	操作码（11位）											S	N	Z	V	C



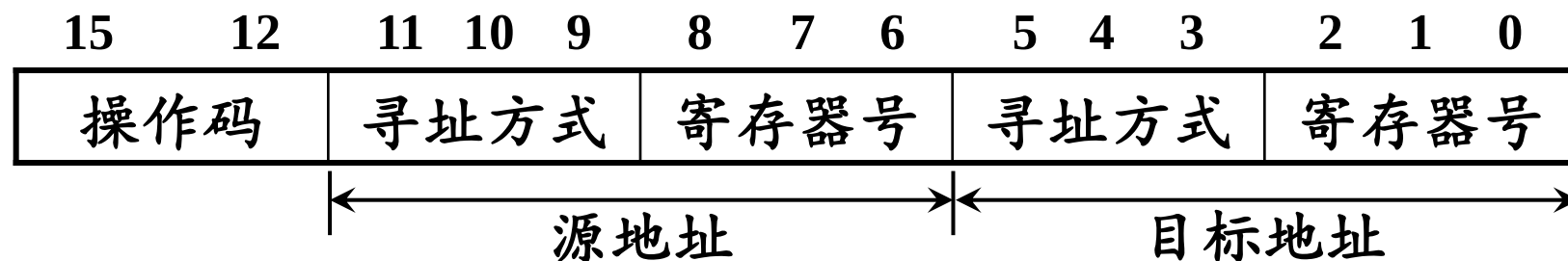
举例：PDP/11系列机寻址方式 (2)

■ 寻址方式

- ◆ 寻址方式指示8种寻址方式，分为直接寻址和间接寻址两大类，还有4种PC寻址方式，是一种隐含寻址方式，以访问R7为标志

■ 寄存器号

- ◆ CPU内有8个寄存器：R0~R7，其中R7为程序计数器，R6为堆栈指示器，指令中用3位表示寄存器编号，形式地址存放在这些寄存器中





例3

一种二地址RS型指令的结构如下所示：

6位		4位	1位	2位	16位
OP	—	通用寄存器	I	X	偏移量D

其中I为间接寻址标志位，X为寻址模式字段，D位偏移量字段，通过I，X，D的组合，可构成下表所示的寻址方式，请写出六种寻址方式的名称。





例3解

■ 解:

	I	X	有效地址E算法	说明	寻址方式
(1)	0	00	$E=D$		直接寻址
(2)	0	01	$E=(PC)+D$	PC为程序计数器	相对寻址
(3)	0	10	$E=(R2)+D$	R2为变址寄存器	变址寻址
(4)	1	11	$E=(R3)$		寄存器间接寻址
(5)	1	00	$E=(D)$		间接寻址
(6)	0	11	$E=(R1)+D$	R1为基址寄存器	基址寻址





例4 (1)

某16位机器所使用的指令格式和寻址方式如图所示，该机有两个20位基址寄存器，四个16位变址寄存器，十六个16位通用寄存器，指令汇编格式中的S（源），D（目标）都是通用寄存器，M是主存中的一个单元。三种指令的操作码分别是MOV（OP）=（A）_H，STO（OP）=（1B）_H，LAD（OP）=（3C）_H。MOV是传送指令，STO为写数指令，LAD为读数指令。

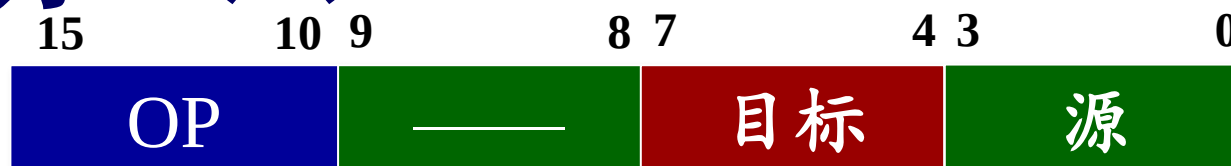
要求：

- (1) 分析三种指令的指令格式与寻址方式
- (2) CPU完成哪一种操作所花时间最短？哪一种操作所花时间最长？第二种指令的执行时间有时会等于第三种指令的执行时间吗？
- (3) 下列情况下每个十六进制指令字分别代表什么操作？其中如果有编码不正确，如何改正才能成为合法指令？





例4 (2)



MOV S, D



STO S, M



LAD M, D

解 (1) 指令1 单字长二地址指令 **RR**型

指令2 双字长二地址指令 **RS**型

M的寻址方式为: 基址+变址+位移量

指令3 双字长二地址指令 **RS**型 **M**为直接寻址





例4 (3)

(2) 指令1执行速度最快

指令2执行速度最慢，因为访存地址要经过计算才能得到

指令3介于前两者之间，比指令2快

(3) ①F0F1 3CD2

$\underbrace{1111\ 0000}_{3C}\ \underbrace{1111}_{15\text{号寄存器}}\ \underbrace{0001\ 0011\ 1100\ 1101\ 0010}_{13CD2}$

把主存13CD2地址单元的内容装载 (LDA) 到15号寄存器





动手做

- 某计算机处理器字长16位，寻址空间为64KB，共有8个16位通用寄存器R0~R7（任一寄存器均可作为基址寄存器或变址寄存器），如果指令系统共有128条双操作数指令，且其中的1个操作数采用寄存器寻址，而另一个操作数的寻址方式有4种：立即寻址、寄存器寻址、基址寻址、变址寻址，并且立即数和位移量均为16位，请设计出该处理器的指令系统格式并说明各字段的含义。

立即寻址	OP (7位)	s/d	Reg (3位)	Mod (00)	R/M (3位)
	立即数 (16位)				

寄存器寻址	OP (7位)	s/d	Reg (3位)	Mod (01)	R/M (3位)
-------	---------	-----	----------	----------	----------

基址寻址	OP (7位)	s/d	Reg (3位)	Mod (10)	R/M (3位)
	位移量 (16位)				

变址寻址	OP (7位)	s/d	Reg (3位)	Mod (11)	R/M (3位)
	位移量 (16位)				





动手做

某指令格式如下所示。

OP	M	I	D
----	---	---	---

其中M为寻址方式，I为变址寄存器编号，D为形式地址。若采用先变址后间址的寻址方式，则操作数的有效地址是

- A $I+D$
- B $(I)+D$
- C $((I)+D)$
- D $((I))+D$



动手做

- 某计算机按字节编址，指令字长固定且只有两种指令格式，其中三地址指令29条，二地址指令107条，每个地址字段为6位，问指令字长至少应该是多少位？
- 解： $6 \times 3 + 5 = 23 \rightarrow$ 指令字长至少为24位，因为按字节编址





4.5 典型指令



指令的分类

- 数据传送指令
- 算术运算指令
- 逻辑运算指令
- 程序控制指令
- 输入输出指令
- 串处理指令
- 特权指令
- 其他指令





数据传送指令

- 完成主存和寄存器之间，或寄存器和寄存器之间的数据传送

主要包括：

- 一般传送指令 **MOV AX, BX**
- 数据交换指令 **XCHG**
- 堆栈操作指令 **PUSH/POP**





算术运算指令

- 完成定点或浮点的算术运算，大型机中有向量运算指令，直接对整个向量或矩阵进行求和、求积运算

主要包括：

- 定点/浮点的加、减、乘、除指令 **ADD**、**SUB**...
- 加1、减1指令 **INC**、**DEC**
- 求反、求负数指令 **NOT**、**NEG**
- 算术移位指令 **SAL**、**SHR**、比较指令 **CMP**
- 十进制加、减运算指令





逻辑运算指令

- 完成无符号数的位操作、代码的转换、判断及运算主要包括：
 - 逻辑或指令 **OR**
 - 逻辑与指令 **AND**
 - 逻辑异或（按位加）指令 **XOR**
 - 逻辑移位指令 **SHR**





程序控制指令

- 程序控制类指令用于控制程序的执行方向，并使程序具有测试、分析与判断的能力。也称为转移指令

主要包括：

- 条件转移指令 JNZ、JC ...
- 无条件转移指令 JMP
- 转子程序指令 CALL
- 返回主程序指令 RET
- 中断返回指令 IRET
-





输入输出指令

- 完成CPU与外设之间的数据传送

主要包括：

- 输入指令 IN
- 输出指令 OUT

注：有的处理器没有专门的输入输出指令





串处理指令

■ 对内存中连续存放的字节或字序列进行处理的指令
主要包括：

- 串传送指令
- 串存储指令
- 串加载指令
- 串比较指令
- 串扫描指令





特权指令

- 特权指令是指具有特殊权限的指令。一般不直接供给用户程序使用，是由系统程序使用的指令
- 这类指令主要完成：
 - ◆ 系统资源的分配和管理
 - ◆ 系统工作方式的设置
 - ◆ 用户访问权限的检测
 - ◆ 段表、页表的修改
 - ◆ 任务的创建和切换
 - ◆ ...





其他指令

主要包括：

- 状态寄存器置位/复位指令 **STC**、**CLC**...
- 测试指令 **TEST**
- 停机指令 **HLT**
- 空操作指令 **NOP**
- 系统控制用的特殊指令 **WAIT**...





基本指令系统的操作（1）

指令类型	操作名称	说明
数据传送	MOV 传送	由源向目标传送字，源和目标是寄存器
	STO 存数	由CPU向存储器传送字
	LAD 取数	由存储器向CPU传送字
	EXC 交换	源和目标交换内容
	CLA 清零	传送全0字到目标
	SET 置1	传送1到目标
	PUS 进栈	由源向栈顶传送字
	POP 出栈	由栈顶向目标传送字





基本指令系统的操作（2）

指令类型	操作名称	说明
算术运算	ADD 加法	计算两个操作数的和
	SUB 减法	计算两个操作数的差
	MUL 乘法	计算两个操作数的积
	DIV 除法	计算两个操作数的商
	ABS 绝对值	求操作数的绝对值
	NEG 变负	改变操作数的符号
	INC 加1	操作数加1
	DEC 减1	操作数减1





基本指令系统的操作 (3)

指令类型	操作名称	说明
逻辑运算	AND 与	按位完成指定的逻辑运算
	OR 或	
	NOT 非	
	EOR 异或	
	TES 测试	测试指令的条件：根据结果设置标志
	COM 比较	对两个操作数进行逻辑或算术比较：根据结果设置标志
	SHI 加1	左（右）移位操作数，一端引入常数
	ROT 减1	左（右）移位操作数，两端环绕
控制转移	JMP 无条件转移	无条件转移：以指定地址装入PC
	JMPX 条件转移	根据测试条件，将指定地址装入PC，或什么也不做
	JMPC 转子	将当前程序控制信息放到一个已知位置：转移到指定地址
	RET 返回	由已知位置内容替代PC和其他寄存器的内容





CISC计算机特点

- 指令系统庞大、指令条数200~300条
- 指令长度不固定，指令格式复杂，寻址方式多。
x86-64指令长度1~15个字节
- 可以访问存储器的指令不受限制
- CPU中通用寄存器数量较少
- 栈密集的过程链接，栈被用来存取过程参数和返回地址
- 各种指令的执行时间相差很大，不利于流水线处理
- 控制器一般采用微程序控制方式，各种指令的使用频率相差较大
- 难以用优化编译生成高效的目标代码





CISC的反思

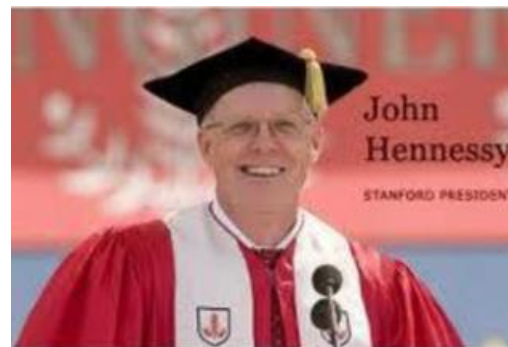
- 复杂指令集计算机（CISC）的指令系统一般多达二三百条
 - ◆ 例如VAX11/780计算机有303条指令，18种寻址方式
 - ◆ 例如Pentium有191条指令，9种寻址方式
- 最常使用的是一些最简单最基本的指令，仅占指令总数的20%，但在程序中出现的频率却占80%





RISC计算机的出现

- RISC思想首先由IBM公司提出，IBM801处理器是公认的体现RISC思想的机器，那时不叫RISC
- 1980/1年伯克利大学的Patterson教授和斯坦福大学的Hennessy教授各自领导的研究组先后提出了RISC概念，并分别设计了RISC I和MIPS处理器芯片
- 相继推出商业芯片
 - ◆ SPARC处理器 SUN公司
 - ◆ MIPS处理器 MIPS公司





典型RISC指令系统的基本特征

型号	指令数	寻址方式	指令格式	通用寄存器数	主频/Mhz
RISC-I	31	2	2	78	8
RISC-II	39	2	2	138	12
MIPS	55	3	4	16	4
SPARC	75	4	3	120-136	25-33
MIPSR3000	91	3	3	32	25
i860	65	3	4	32	50
PowerPC	64	6	5	32	





RISC计算机特点

- 选取使用频率最高的一些简单指令，指令数量少（<100）
- 指令长度固定，指令格式种类少，寻址方式简单
- 只有load / store指令访问存储器，其余指令的操作都在寄存器之间进行
- CPU中通用寄存器数量较多，可减少访存次数
- 寄存器密集的过程链接，寄存器用来存放过程参数和返回地址，有些过程能完全避免内存引用
- 大部分指令都能在一个机器周期内完成，采用流水线技术
- 采用硬布线控制逻辑，控制器简单，留出更多芯片面积用于增大Cache容量
- 编译器技术优化





Powe PC指令类型

指令32位等长

- 整数算术运算 / 逻辑运算 / 移位指令
- 浮点运算指令
- 取数 (LOAD) / 存数 (STORE) 指令
- 条件寄存器指令
- 控制转移指令

详见第六版教材, p139





ARMv8-A架构支持的指令集

■ AArch64执行状态

- ◆ 只能使用A64指令集
- ◆ 指令为32位等长指令字

■ AArch32执行状态

◆ A32指令集:

- 对应ARMv7架构及其之前的ARM指令集
- 32位等长指令字

◆ T32指令集

- 对应ARMv7架构及其之前的Thumb/Thumb-2指令集
- 16位和32位可变长指令字结构





ARM寻址方式

■ ARMv8 A64处理器六种寻址方式

- ◆ 立即数寻址
- ◆ 寄存器寻址
- ◆ 寄存器间接寻址
- ◆ 基址寻址
- ◆ 堆栈寻址
- ◆ 相对寻址





ARM A64指令集的指令分类

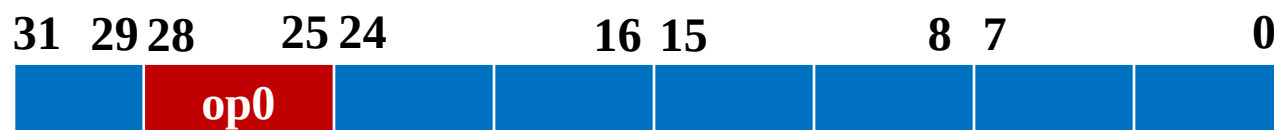
指令类型	说明
分支指令	条件跳转、无条件跳转 (#imm 、 register)指令
异常发生指令	系统调用类指令 (SVC 、 HVC 、 SMC)
系统寄存器指令	读写系统寄存器，如： MRS 、 MSR 指令可操作 PSTATE 的位段寄存器
数据处理指令	包括各种算数运算、逻辑运算、位操作、移位 (shift) 指令
load/store 访存指令	load/store （单寄存器、寄存器对儿、多寄存器等）





A64指令集编码 (1)

■ A64 instruction set encoding



字段编码 op0	编码分组或指令类型
0000	保留 (Reserved)
0001	未分配 (Unallocated)
0010	SVE (Scalable Vector Extensio) 指令
0011	未分配 (Unallocated)
100x	数据处理——立即寻址
101x	分支、异常发生和系统指令
x1x0	Loads and Stores内存访问指令
x101	数据处理——寄存器
x111	数据处理——浮点与SIMD指令





A64指令集编码 (2)

■ add/subtract (immediate) instruction set encoding



Decode fields	Instruction
sf op S	
0 0 0	ADD (immediate) - 32-bit
0 0 1	ADDS (immediate) - 32-bit
0 1 0	SUB (immediate) - 32-bit
0 1 1	SUBS (immediate) - 32-bit
1 0 0	ADD (immediate) - 64-bit
1 0 1	ADDS (immediate) - 64-bit
1 1 0	SUB (immediate) - 64-bit
1 1 1	SUBS (immediate) - 64-bit

op	Instruction
00x	PC-rel. addressing
010	Add/subtract(immediate)
011	Add/subtract(imm., tags)
100	Logical (immediate)
101	Move wide (immediate)
110	Bitfield
111	Extract

例: add x16, x16, #0xff8 @ (x16) + 0xff8 $\Rightarrow$ x16

机器码为: 1001 0001 0011 1111 1110 0010 0001 0000

Rn:源寄存器

Rd:目标寄存器

imm12:12位立即数

S:为1时指令执行将影响

PSTATE中的标志位NZCV





本章小结

- 指令格式
- 指令寻址方式
 - ◆ 顺序寻址方式、跳跃寻址方式
- 数据寻址方式
 - ◆ 隐含寻址、立即寻址、直接寻址、间接寻址、寄存器寻址、寄存器间接寻址、偏移寻址、段寻址、堆栈寻址
- 典型指令
 - ◆ 数据传送指令、算术运算指令、逻辑运算指令、程序控制指令、输入输出指令、串处理指令、特权指令、其他指令
- CISC、RISC计算机特点

